Backpropagation - Part 3: The Why

1 The Intuition Behind the Algorithm

Think of backpropagation like **learning from feedback**.

When the model makes a prediction:

- It checks how far off it is (the **error**).
- Then, instead of guessing random adjustments, it calculates **how to change each weight** so the next prediction is a little better.

It's a **smart trial-and-error process**, the model uses math (derivatives) to figure out the most efficient direction to move in.

2 Concept of Gradient

A **gradient** is just the *slope* or *direction of steepest change* in the loss function.

Imagine the loss function like a mountain:

- The **height** = error (loss).
- The **goal** = reach the lowest valley (minimum loss).
- The **gradient** = tells you which direction to go downhill fastest.

So, in backpropagation:

- Gradient shows how much each weight contributes to the loss.
- The algorithm **moves weights downhill** (reducing loss) by taking small steps in the opposite direction of the gradient.

$$w = w - \eta \cdot \frac{\partial L}{\partial w}$$

Here:

- w = weight
- η = learning rate (step size)
- $\frac{\partial L}{\partial w}$ = gradient

3 Concept of Derivative

A **derivative** measures how one thing changes with respect to another.

In our case, how the **loss** changes when we change a **weight** slightly.

If derivative = positive → increasing weight increases loss.

If derivative = negative → increasing weight decreases loss.

So we move in the **opposite direction** of the derivative to reduce loss.

4 Concept of Minima

When you plot the loss function on a graph, it often looks like a bowl:

- The **lowest point** = **minimum loss** (best possible model).
- Backpropagation with gradient descent **iteratively moves** the model toward that minimum.

Two key terms:

- **Global Minimum** → best possible solution.
- **Local Minimum** → smaller dip that's not the best but still low.

The goal is to reach the *global minimum* (lowest possible loss).

5 Intuition of Backpropagation

Here's what's really happening inside:

1. Forward pass → make predictions and compute loss.
2. Backward pass → compute gradients (how each weight affected loss).
3. Adjust weights → take a small step downhill (gradient descent).
4. Repeat many times → gradually approach minimum error.

It's like **learning to ride a bike**:

- At first, you fall (big loss).
- You adjust slightly (update weights).
- Repeat → you get better → fewer mistakes (loss decreases).

6 Demo

The video likely shows how loss decreases with each training step.

You can **visually explore this** using Google's **Backpropagation Interactive Tool**:

Google Backpropagation Tool

How to use it:

- You'll see a small network diagram with inputs, hidden layers, and outputs.
- As you move the sliders or step through the training:
    - It shows **forward propagation** (how signals move forward).
    - Then **backpropagation** (how errors move backward).
    - Watch how **weights update** and **loss reduces** visually.

It's one of the best tools to **see gradients and convergence in action**.

7 What is Convergence?

Convergence means the model has learned enough — loss stops decreasing significantly.

When the model:

- Keeps making small adjustments,
- But the overall error doesn't improve anymore →

  -> It has **converged**.

Think of it as "reaching the bottom of the valley".

If the learning rate is:

- Too **high** → the model jumps around, never settles.
- Too **low** → training is very slow.

  So choosing the right learning rate helps reach convergence efficiently.

➢ In Summary

Concept	Meaning	Key Intuition
Gradient	Direction to reduce loss	Slope of the loss curve
Derivative	Rate of change	Measures sensitivity
Minima	Lowest point on loss curve	Goal of training
Convergence	When training stabilizes	Model has learned enough
Backpropagation	Core learning process	Uses gradients to adjust weights

---

➢ Final Thought:

Backpropagation + Gradient Descent = The engine that drives all deep learning.

If forward propagation is the model's *thinking*,

then backpropagation is the model's *learning from mistakes*.
